

Name: _____

Class: _____



Mr. Rawson • GCSE Computer Science

Year 10 2026

Hex in Colour

Why anyone bothered inventing hexadecimal in the first place.



#FFDE34

the yellow of every emoji

F F D E 3 4
 RED GREEN BLUE
 $FF = 255$ $DE = 222$ $34 = 52$
 $15 \times 16 + 15$ $13 \times 16 + 14$ $3 \times 16 + 4$

Two hex digits per channel. **Two hex digits are one byte**, so each channel runs **0 to 255** — and #FFDE34 is **rgb(255, 222, 52)**.

The 34 is the trap, and it is the point of this sheet. FF and DE have letters in them, so nobody is fooled — you can see they are not ordinary numbers. 34 has no letters. It looks like thirty-four. It is fifty-two.

Any hex pair without a letter in it is a booby trap: 10 is 16 · 20 is 32 · 80 is 128 · 99 is 153

How many colours is that?

Each channel is one byte, so each has **256** levels (0 to 255).

$$256 \times 256 \times 256 = 16\,777\,216$$

The mistake to avoid: $256 + 256 + 256 = 768$.

The channels multiply. Every red can be paired with every green, and every one of those with every blue.

Where you have seen this already

Three bytes is **24 bits**. That is why the box of every television and every phone says **24-bit colour**, or “**true colour**” — and why it claims **16.7 million colours**.

That number 24 is the colour depth, and it is straight off the exam paper:

$$\text{file size} = \text{width} \times \text{height} \times \text{colour depth}$$

Look up and predict

First, without converting: which is exactly halfway?

Convert each pair to denary. **Watch every pair that has no letter in it.** The last two are yours — find a colour you like, write its code, and colour the swatch in.

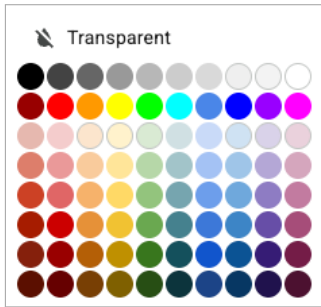
Colour	Hex code	Red	Green	Blue
	#000000			
	#FFFFFF			
	#FF0000			
	#808080			
	#1E90FF			
	#00A550			
colour it in				
colour it in				

Six hex digits = six nibbles = three bytes, one for each channel.

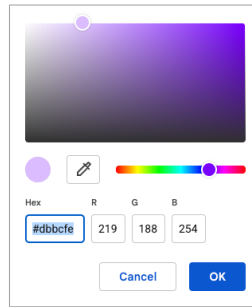
A colour code is three byte strips side by side.

How many colours are there *really*?

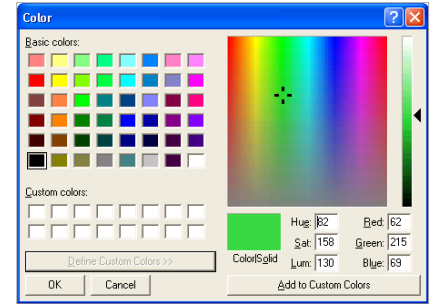
Two colour pickers, and a question with no agreed answer.



Google Docs, the default
80 colours



... and its “custom”
tab
16 777 216 colours



Windows, the old Color dialog
48 to point at, 16 777 216 to type

Why only 80? Because a grid you point at has to fit on a screen. **80 is a number about fingers and screens, not about colour.** The Hex box is the escape hatch — and look what it wants: **#dbbcfe**, with **R 219, G 188, B 254** beside it. **Exactly the sheet you did on the other side.** The old Windows dialog is the same idea drawn differently — a few dozen to point at, a field to drag around, and the numbers underneath, which are the part that is actually true.

16 777 216 is not a measurement. It is arithmetic. $256 \times 256 \times 256$. Nobody had to look at anything to get it, and there is nothing to argue about.

How many colours a person can tell apart is a measurement — and measurements of it disagree wildly:

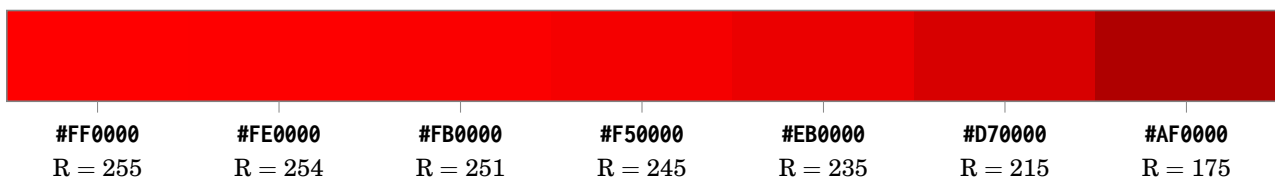
Estimate	Who, and when	How they got it
~10 000 000	Judd, 1939	the famous one — method never published
~1 900 000	Nickerson & Newhall, 1943	counting cells in a colour solid
~2 300 000	Pointer & Attridge, 1998	counting unit cubes in CIELAB
~350 000	Seymour, 2013	500 million simulated spectra, modern formula

That is a factor of nearly thirty — and the biggest number is the one with the least evidence behind it. The honest answer is **nobody knows, and it depends what you mean by “different”**. What everyone agrees on: **a lot fewer than 16.7 million.**

Prove it yourself

no equipment, about thirty seconds

Seven reds; only the **red** channel changes, and only downwards. Left to right — **where do you first see a change?**



First code where you can see a difference: # _____ 0000

Now think about the first step. FF to FE is a change of **one** — the smallest change the code can express. If you cannot see it, and almost nobody can, then there are colours among the 16.7 million that your eye files as the same colour.

So why use 24 bits, if the eye cannot use them all? Because the computer is not storing what you can *see* — it is storing an **address**, and an address has to be exact. And because 8 bits per channel is **one byte per channel**, the tidiest amount a machine can hold.